



UNIVERSIDAD
NACIONAL
DE LA PLATA

Visado 7

Wallet Pattern

Pedro Spadari - 21586/8

Pasos aplicados en el refactoring

1. Crear interfaz MnemonicSeedPhraseHandler.
2. Crear clase MnemonicSeedPhraseProvider que implementa MnemonicSeedPhraseHandler.
3. Mover métodos de App a MnemonicSeedPhraseProvider:
 - a. boolean isValid(seedPhrase: String): boolean.
 - b. String[] createMnemonicAsWords(): String[].
 - c. String createMnemonicString(): String.
4. Renombrar métodos de MnemonicSeedPhraseHandler (y su implementación en MnemonicSeedPhraseProvider):
 - a. createMnemonicString() a randomMnemonic().
 - b. createMnemonicAsWords() a randomMnemonicWords().
5. Crear relación en App:
 - a. Agregar variable de instancia mnemonicSeedPhraseHandler: MnemonicSeedPhraseHandler.
6. Crear interfaz KeyStorer.
7. Crear clase SeedManager que implementa KeyStorer. Usa el patrón Singleton retornando una instancia desde el método defaultManager().
8. Mover métodos de App a SeedManager:
 - a. static KeyStore loadKeyStoreFromFile(pathToFile: String, keystorePassword: String): KeyStore.
 - b. static void saveKeyStoreOnFile(keyStore: KeyStore, pathToFile: String, keystorePassword: String).
 - c. static String getKey(keyStore: KeyStore, entryPassword: String, alias: KeyStoreEnum): String.
 - d. static void writeToKeyStore(keyStore: KeyStore, entryPassword: String, alias: KeyStoreEnum, password: String).
 - e. getKey(): String.
 - f. saveKey(key: String).
 - g. Seed getSeed(): Seed.
 - h. seedPhase (String seedBytes).
9. Crear relación en App:
 - a. Agregar variable de instancia keyStorer: KeyStorer.
10. Crear interfaz WalletSynchronizer.
11. Crear clase ConcreteWalletSynchronizer que implementa WalletSynchronizer.
12. Mover métodos de App a ConcreteWalletSynchronizer:
 - a. void start().
 - b. void stop().
 - c. boolean isSynchronizing().
 - d. SyncStatus getSyncStatus(): SyncStatus.
 - e. SyncResult sync(): SyncResult.
 - f. Transaction[] listTransactions(): Transaction[].
 - g. List<Transaction> pendingTransactions(): List<Transaction>.
 - h. List<Transaction> receivedTransactions(): List<Transaction>.
 - i. List<Transaction> sentTransactions(): List<Transaction>.
 - j. UnifiedAddress[] getAddresses(): UnifiedAddress[].



- k. `SendResult sendTo(address: String, zatoshis: long, memo: String): SendResult.`
- l. `Balance getBalance(): Balance.`
- 13. Crear relación en App:
 - a. Agregar variable de instancia `walletSynchronizer: WalletSynchronizer.`
- 14. Crear clase `WalletInitializer.`
- 15. Mover métodos de App a `WalletInitializer:`
 - a. `Seed createNewWallet(dataDir: String): Seed.`
 - b. `Seed createNewWallet(dataDir: String, seedPhrase: String[]): Seed.`
 - c. `Seed createNewWallet(dataDir: String, keywords: String[]): Seed.`
 - d. `void restoreWallet(dataDir: String, seedPhrase: String, birthday: Integer).`
 - e. `Seed createNewWalletUsingRandomSeed(walletPath: String): Seed.`
 - f. `Seed createNewWalletFromKeyStore(walletPath: String, pathToKeyStore: String, keystorePassword: String, dataPassword: String): Seed.`
 - g. `boolean walletExists(path: String): Boolean.`
- 16. Mover constructor de App a constructor de `WalletInitializer.`
- 17. Crear el método `initWallet(walletPath: String, backendPath: String)` en App que instancia al `WalletInitializer.`